Meridian Backend — Complete Code Walkthrough

`backend/` — Strapi 5.60 on Node.js 24 (TypeScript). One process serving the REST API, admin panel, auth, media pipeline, and all custom business logic.

1. What this "backend" actually is

Strapi is a Node.js application built on **Koa** (HTTP layer) and **Knex** (SQL query builder). There is no separate server framework — every file in `backend/src/` either declares content (schemas), extends Strapi's generated machinery (controllers/routes/services), or runs custom logic (bootstrap, lifecycles, middleware). At boot, Strapi scans `src/api/*` and auto-registers REST endpoints for each content type; our custom files hook into that same pipeline.

> **Request lifecycle:** HTTP → Koa middlewares (CORS, rate-limit, auth) → router → controller → document service → Knex → database → JSON response.

2. Folder map

```
backend/
├── config/                 — environment-driven app configuration
├── src/
│   ├── index.ts            — bootstrap entry: runs setup on every start
│   ├── api/                — one folder per content type (6 total)
│   │   └── <type>/
│   │       ├── content-types/<type>/schema.json   ← fields & relations
│   │       ├── controllers/<type>.ts              ← request handlers
│   │       ├── routes/<type>.ts                   ← auto CRUD router
│   │       ├── routes/custom-*.ts                   ← custom endpoints
│   │       └── services/<type>.ts                 ← DB query layer
│   ├── bootstrap/          — boot-time setup (perms, tokens, admin, seed)
│   ├── middlewares/        — api-rate-limit.ts
│   ├── components/         — shared.social-links (reusable field group)
│   └── admin/              — admin-panel customizations (unused examples)
└── public/uploads/         — local media storage (dev; Cloudinary in prod)
```

3. config/ — app configuration

FILE	PURPOSE
`database.ts`	Env-driven DB: `DATABASE_CLIENT=sqlite|postgres`. Supports `DATABASE_URL` connectionString + SSL flag. SQLite → `.tmp/data.db`; prod → Neon Postgres. No code change needed between environments.
`middlewares.ts`	Koa middleware stack: CORS locked to `CORS_ORIGIN`, security headers, body parsing, and registers our `api-rate-limit` middleware.

`plugins.ts`	Plugin config: users-permissions (JWT/refresh, httpOnly sessions) + upload provider — **switches to Cloudinary when `CLOUDINARY_NAME` is set**, else local disk. Also the upload MIME allow/deny lists (blocks SVG, executables, scripts).
`server.ts`	Host/port, `APP_KEYS` (session signing).
`admin.ts`	Admin panel auth secrets (`ADMIN_JWT_SECRET`).
`api.ts`	REST API defaults (pagination limits).

4. Content types — `src/api/*/schema.json`

Each schema declares fields; Strapi generates the REST API, DB table, admin forms, and validation from it.

TYPE	KEY FIELDS	NOTES
article	title, slug (uid), excerpt, content (Blocks rich text), featuredImage, thumbnailImage, ogImage, seoTitle/Description, canonicalUrl, featured, publishedDate, readingTime, viewCount	Core entity. draftAndPublish on → public API only exposes published entries. Relations: → category, → author, ↔ tags.
category	name, slug, description, image, seoTitle, seoDescription	Taxonomy + SEO on landing pages.
author	name, slug, avatar, biography, designation, socialLinks (component)	Profile data for author pages.
tag	name, slug	Many-to-many with articles.
newsletter-subscriber	email (unique), subscribedAt, status enum (active/unsubscribed)	draftAndPublish off — created via custom endpoint only.
contact-message	name, email, subject, message, submittedAt, status enum (new/read/archived)	Admin-inbox model; public can only submit.
website-setting (single type)	siteName, siteDescription, logo, favicon, socialLinks, contactEmail, footerContent, defaultSeoTitle/Description, defaultOgImage	One global document — navbar/footer/SEO defaults.

Component: `src/components/shared/social-links.json` — reusable twitter/linkedin/github/instagram group embedded in author + settings schemas.

5. How the REST API gets generated

For each content type, three factory calls produce the standard surface:

```
// routes/article.ts        →  GET/POST/PUT/DELETE /api/articles[/:id]
export default factories.createCoreRouter('api::article.article');

// controllers/article.ts   →  find / findOne / create / update / delete
export default factories.createCoreController('api::article.article', ({strapi}) => ({ /* overrides */

// services/article.ts      →  query layer used by controllers
export default factories.createCoreService('api::article.article');
```

Public access is then gated by the Public role's permissions (see §7), so e.g. articles are publicly `find`-able but contact-messages are not listable at all.

6. Custom endpoints (the business logic)

6.1 View counter — `routes/custom-article.ts`

```
POST /api/articles/:documentId/view   (auth: false)
```

Controller `incrementView`: fetches the published document, updates `viewCount + 1` via the document service, returns the new count. Not-found → 404. Fires `afterUpdate` → revalidates the frontend.

6.2 Newsletter — `routes/custom-newsletter-subscriber.ts`

```
POST /api/newsletter-subscribers/subscribe   (auth: false)
```

Controller `subscribe`: validates email regex + ≤254 chars → 400 on bad input; looks up existing by email → re-activates if `unsubscribed`, no-op if active (**idempotent**); else creates with `subscribedAt: now`.

6.3 Contact form — `routes/custom-contact-message.ts`

```
POST /api/contact-messages/submit   (auth: false)
```

Controller `submit`: validates name ≤120, email regex, message ≤5000 → creates entry with `status: 'new'`, returns only a documentId (no echo of submitted PII).

6.4 Standard admin CRUD

Core routers also expose full CRUD on every type — but the Public role only gets `find` / `findOne` on content types and the three custom mutations. Write access requires the admin panel session or a scoped API token.

7. src/bootstrap/ — runs on every boot (src/index.ts)

```
async bootstrap({ strapi }) {
  await configurePublicPermissions(strapi);
```

```
  await ensureFrontendToken(strapi);
  await ensureAdminUser(strapi);
  await seedDemoData(strapi);
}
```

MODULE	WHAT IT DOES
permissions.ts	Idempotently grants 12 actions to the Public role: `find/findOne` on article/category/author/tag/website-setting + `incrementView` + `subscribe` + `submit`. Everything else is admin/token only.
api-token.ts	Ensures a custom-scoped Content API token (`frontend-content-api`) exists with exactly the frontend's needed permissions; writes the plaintext key to `.frontend-api-token` (gitignored) once — Strapi only reveals it at creation.
admin-user.ts	If `ADMIN_EMAIL` + `ADMIN_PASSWORD` envs are set: creates a super-admin when none exists for that email, or resets its password — admin recovery on hosts without shell access (Render free tier).
seed.ts	Empty DB → seeds settings, 4 categories, 3 authors, 8 tags, 9 articles (+ generated PNG covers/avatars via the upload service). `EXTRA_ARTICLES` list upserts 6 more by slug every boot — idempotent, skips existing.

8. Lifecycles — article/lifecycles.ts

HOOK	EFFECT
`beforeCreate / beforeUpdate`	Extracts text from Blocks content → `readingTime` (words ÷ 200 wpm, min 1); defaults `publishedDate` to now.
`afterCreate / afterUpdate / afterDelete`	`notifyFrontend()` → fire-and-forget POST to `FRONTEND_URL/api/revalidate?secret=…` → Next.js clears ISR cache → publish goes live instantly. Publish/unpublish rides on afterUpdate.

9. Middleware — src/middlewares/api-rate-limit.ts

In-memory per-IP limiter on `/api/*` : `RATE_LIMIT_MAX` requests per `RATE_LIMIT_WINDOW_MS` (default 120/60s) → 429 + warn log when exceeded. Periodic sweep prevents unbounded map growth. Best-effort single-instance — production traffic limiting should also live at CDN/edge.

10. Environment variables

VAR	PURPOSE
`DATABASE_CLIENT / DATABASE_URL / DATABASE_SSL / DATABASE_FILENAME`	DB selection — sqlite file locally, Neon URL + ssl in prod

`APP_KEYS, API_TOKEN_SALT, ADMIN_JWT_SECRET, TRANSFER_TOKEN_SALT, JWT_SECRET, ENCRYPTION_KEY`	Strapi crypto secrets (fresh values generated for prod)
`CLOUDINARY_NAME / KEY / SECRET`	Presence switches upload provider local → Cloudinary
`CORS_ORIGIN`	Allowed browser origin(s)
`FRONTEND_URL, REVALIDATE_SECRET`	Lifecycle → Next.js revalidate POST
`ADMIN_EMAIL, ADMIN_PASSWORD`	Optional super-admin provisioning/reset
`RATE_LIMIT_WINDOW_MS, RATE_LIMIT_MAX`	Public API throttle

11. Security model recap

- **Public role:** read published content only + 3 validated mutations; no draft/unpublished/Pii reads (contact messages, subscribers are not publicly listable).
- **Scoped API token:** least-privilege permissions, server-side env only.
- **Validation:** every custom endpoint validates + bounds input lengths.
- **Upload guard:** MIME allow/deny lists; no SVG/executables.
- **Rate limiting + CORS lockdown** on the public surface.
- **Secrets:** all in env/gitignored files; production uses fresh values.